

You have used these 11 features for three lectures

Here is bike-day.csv, exactly as it sits on disk:

instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
1	2011-01-01	1	0	1	0	6	0	2	0.344	0.364	0.806	0.160	331	654	985
2	2011-01-02	1	0	1	0	0	0	2	0.363	0.354	0.696	0.249	131	670	801



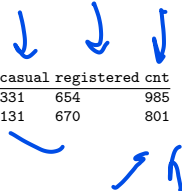
50 → 15

RFE

You have used these 11 features for three lectures

Here is bike-day.csv, exactly as it sits on disk:

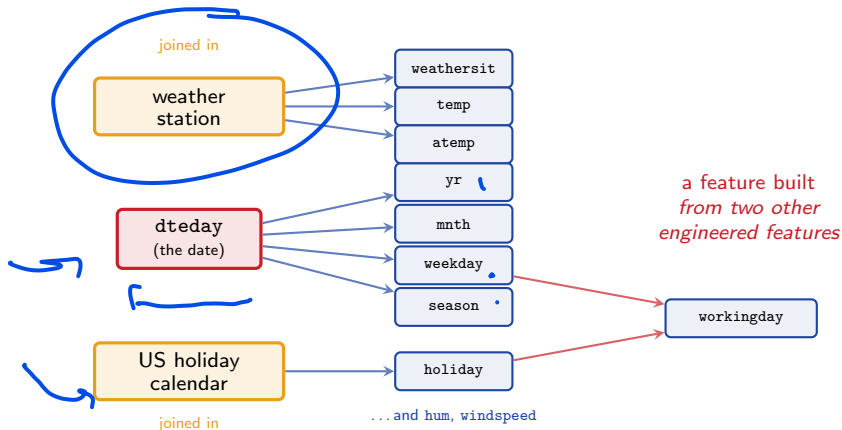
instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
1	2011-01-01	1	0	1	0	6	0	2	0.344	0.364	0.806	0.160	331	654	985
2	2011-01-02	1	0	1	0	0	0	2	0.363	0.354	0.696	0.249	131	670	801



**Which of these columns came off a sensor or a calendar,
and which ones did a human build?**

Take a guess before the next slide. Most people say “most of them are raw.”

Exactly one column is raw



The design matrix you have been interpreting for three lectures is a **worked feature-engineering example**. Everything in this deck is already on this slide.

And one thing that should never have shipped

$$\text{casual} + \text{registered} = \text{cnt}$$

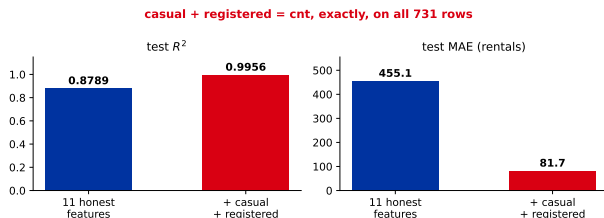
Exactly. On **all 731 rows**.

$$331 + 654 = 985 \quad \checkmark$$

$$131 + 670 = 801 \quad \checkmark$$

Two columns shipped in the dataset **add up to the target**. Fit on them and you get a near-perfect model that has learned nothing about bicycles.

This is **target leakage**, and it is the single most common way a model lies to you. It will come back three more times in this deck — every time, it enters through a *feature*.



Outline

What and why

From the same row

From many rows

From outside the data

Generating candidates at scale

Pitfalls

Three places a feature can come from

The same row, many rows, or outside your table entirely.

Preprocess, engineer, select — three different verbs

- ✓ ▶ **Preprocessing** (lecture 3) — *fix* the data. Same columns, cleaner.
- ▶ **Feature engineering** (this lecture) — *create* columns. More columns.
- ▶ **Feature selection** (next lecture) — *choose* columns. Fewer columns.

```
df["hum"] = df["hum"].fillna(df["hum"].median()) # preprocess: fix
df["discomfort"] = df["hum"] * df["windspeed"] # engineer
X = X[["temp", "yr", "discomfort"]] # select: choose
```

They get confused constantly, and the difference matters: preprocessing is *obligatory*, engineering is a *bet*, and selection is a *decision*.

Predict-first: where does a machine-learning project's time go?

A project has stages: collect data, explore, preprocess, engineer features, model, tune, evaluate, deploy.

Predict-first: where does a machine-learning project's time go?

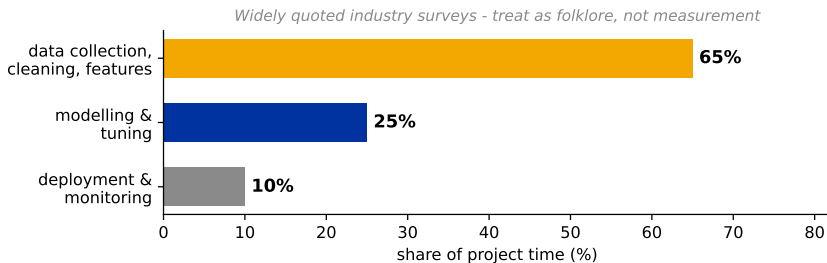
A project has stages: collect data, explore, preprocess, engineer features, model, tune, evaluate, deploy.

What fraction goes to data and features, versus modelling and tuning?

Predict-first: where does a machine-learning project's time go?

A project has stages: collect data, explore, preprocess, engineer features, model, tune, evaluate, deploy.

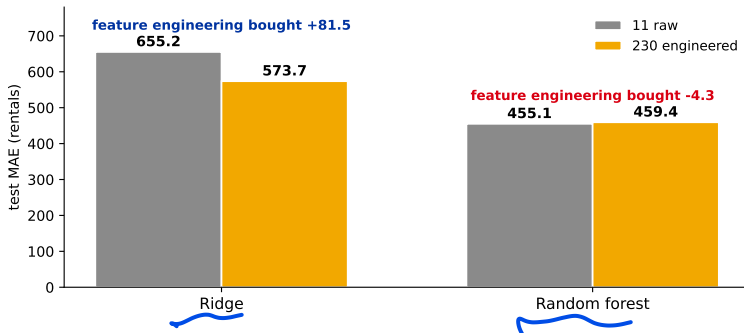
What fraction goes to data and features, versus modelling and tuning?



Students almost always guess the reverse. **But be careful with this number:** it comes from vendor surveys (CrowdFlower 2016, Anaconda 2020), not from research. Nobody has measured it rigorously. Treat it as *folklore that points in the right direction*.

Does feature engineering still matter in 2026?

The classic line is “good features beat a better algorithm.” That was the 2015 consensus. Let us test it on *our own data* — same features, two model families:



The same 219 engineered features are worth **+81.5** mean absolute error (MAE) to a linear model and **-4.3** to a forest — it got slightly *worse*. The forest was already deriving them internally, through nested splits (lecture 22).

So when is it worth the effort?

Feature engineering still wins decisively where the model **cannot get there alone**:

- ▶ **External joins.** No model invents the weather. Slide 2 is the proof.
- ▶ **Domain knowledge.** A ratio the model would need far more data to discover.
- ▶ **Small data.** You cannot afford to spend rows teaching the model what you already know.
- ▶ **Leakage-free time features.** Building these *correctly* is a modelling decision, not an automation problem.
- ▶ **Linear and distance-based models**, which construct nothing on their own — as the previous slide just showed.

The thesis for the whole chapter: *generate cheaply, select ruthlessly.* It is fine to mass-produce candidates — as long as the next lecture prunes them.

When the model does this for you

Hand-engineering is what you do when the model *cannot* do it itself.

- ▶ **Images:** convolutional networks learn edge and texture detectors. Nobody hand-codes those any more.
- ▶ **Text:** transformers learn token representations. Same story.
- ▶ **Tabular:** genuinely mixed. Trees derive interactions; nothing derives your company's business logic.

The trend of the last decade is features moving *from* the engineer *into* the model — fastest for images and text, slowest for tabular data.

We come back to this properly in the neural-network chapter, where “learning a representation” becomes the whole point.

The honest summary: feature engineering is not dead — it moved. It is worth less on the parts a model can derive, and just as much on the parts it cannot.

The three sources of a new feature



1 - 100
4343

From the same row

one observation in, one feature out

- ▶ transformations: log, power, bins
- ▶ ratios, differences, products
- ▶ interactions

bike: yr, mnth, weekday, season — all squeezed out of dteday

From many rows

a group of observations in, one feature out

- ▶ group-by aggregations
- ▶ ranks within a group
- ▶ lags and rolling windows

bike: nothing — and that is a gap we will fill

From outside the data

joins to another source

- ▶ calendar and holidays
- ▶ weather
- ▶ geography, demographics

bike: holiday, and all five weather columns

18.2i
25.4
[10.4]

WU

Domain knowledge cuts across all three and is the biggest single multiplier. The next three sections take these one at a time.

One row in, one feature out

The cheapest features to build — and the ones a linear model cannot build for itself.

Ratios and differences



Often the most predictive column in a table is not in the table — it is a *rate*.

Bmi

Canonical examples:

lb in

- ▶ body mass index = weight / height²
- ▶ loan-to-income = loan / income
- ▶ debt-to-equity, price per m², clicks per impression

Why they work:

- ▶ scale-invariant — 30k on 60k is the same risk as 3k on 6k
- ▶ one column instead of two
- ▶ a linear model *cannot* construct them

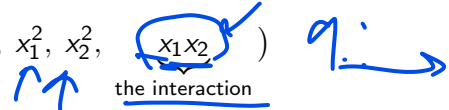
Before you subtract two columns, check they are in the same units. On bike, temp and atemp are *both* normalised — but **differently**: $\text{temp} = (T+8)/47$ and $\text{atemp} = (T_a+16)/66$ (lecture 22). So $\text{atemp} - \text{temp}$ is not a “feels-like gap” — it is an arbitrary mixture of two different scales.

temp / 47, atemp / 66

Interactions: what a linear model cannot see

From lecture 22: a **main effect** is what one feature does on its own; an **interaction** is when the effect of one feature *depends on* another.

Polynomial regression already did this for a single feature (lecture 2). Extend it to pairs:

$$\phi(\mathbf{x}) = (1, \underline{x_1}, \underline{x_2}, \underbrace{x_1^2, x_2^2}_{\substack{\uparrow \uparrow \\ \text{the interaction}}}, \underbrace{x_1 x_2}_{\substack{\text{the interaction}}})$$


The product term is what lets a linear model say “*warm weather matters more at the weekend*”.

Scale before you expand. If x runs to the hundreds then x^3 runs to the millions, and a Ridge or Lasso penalty lands almost entirely on the low-order terms. Put `StandardScaler` before `PolynomialFeatures` in the pipeline, always.

Predict-first: who needs to be told about the interaction?

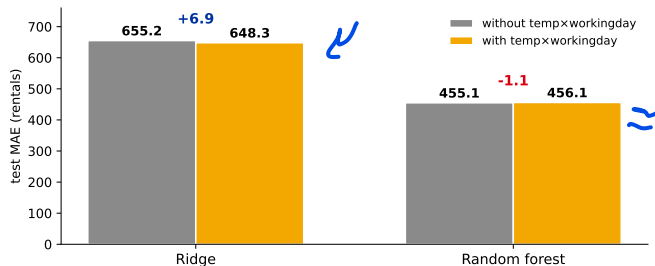
You believe warm weather drives *leisure* riding, so temp should matter more on non-working days. You add $\text{temp} \times \text{workingday}$ explicitly.

Does it help (a) Ridge regression, (b) a random forest?

Predict-first: who needs to be told about the interaction?

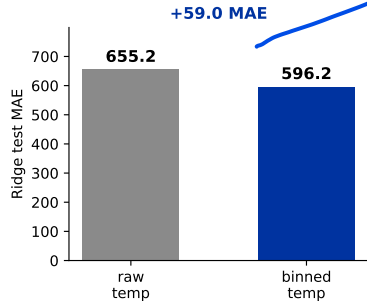
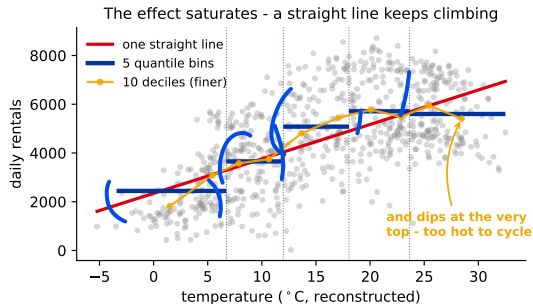
You believe warm weather drives *leisure* riding, so temp should matter more on non-working days. You add temp × workingday explicitly.

Does it help (a) Ridge regression, (b) a random forest?



Ridge: yes (+6.9). Forest: no (-1.1, marginally worse) — it already found that interaction by splitting on workingday *below* a split on temp. **Polynomial and interaction features are a linear-model crutch, not a universal good.** Adding them to a tree ensemble is wasted compute.

Binning: when a straight line is the wrong shape



Three ways to cut a continuous feature:

- **equal-width** — same range per bin
- **equal-frequency (quantile)** — same count per bin
- **supervised** — splits chosen to separate y (this is what a tree does)

Bin when: the effect is non-monotonic *and* the model is linear.

Never bin for trees — they bin implicitly, and you would only be throwing away resolution.

Rentals climb, then **flatten** above $\sim 18^{\circ}\text{C}$ and dip at the very top. A straight line has to keep climbing — so it is wrong in the middle *and* at the top. Five quantile bins: +59.0 MAE, the biggest single win in this deck.

Already covered — deliberately not repeated

Two same-row techniques belong here in the taxonomy, but you already have them from **lecture 3 (preprocessing)**:

Skew transforms

- ▶ `np.log1p` for right-skewed positives
- ▶ `PowerTransformer` (Yeo-Johnson, Box-Cox)
- ▶ invert with `np.exp` to report error in real units

Datetime components

- ▶ `.dt.year`, `.dt.month`, `.dt.dayofweek`, `.dt.hour`
- ▶ weekend and holiday flags
- ▶ **cyclic** sin / cos encoding, so hour 23 sits next to hour 0

If you only remember one thing from that lecture: `dteday` on slide 2 became *four* columns this way. That is the single highest-yield transformation in tabular machine learning, and it costs one line of pandas.

A group of rows in, one feature out

Powerful, and the place leakage most often gets in.

Group-by aggregations

For each row, attach a summary of *its group*. Enormously effective on tabular data.

```
g = df.groupby(["season", "workingday"])["cnt"]

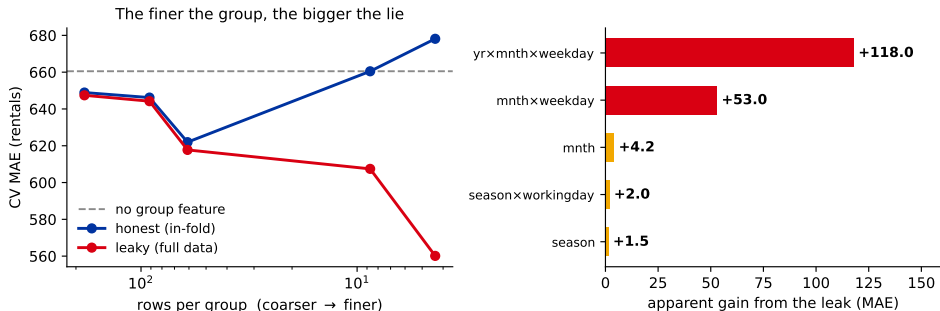
df["grp_mean"] = g.transform("mean")      # what is typical for days like
    this?
df["grp_std"] = g.transform("std")        # how variable are days like this?
df["grp_count"] = g.transform("count")    # how much evidence do we have?
df["rank_in_grp"] = g.rank(pct=True)     # where does THIS day sit in its
    group?
df["vs_grp"] = df["cnt"] / df["grp_mean"]
```

The last two are usually the strongest: a raw value says *how much*, but a rank or ratio-to-group says *how unusual*, which is often what actually predicts.

Look closely at those five lines. **Every one of them touches** cnt — the target. That is fine, and it is also a trap.

The leak hiding in the previous slide

Compute the group mean on the **whole dataset** and then split, and each row's own target has already contributed to its own feature.



Leakage severity scales with how precisely the group identifies the row. At 183 rows per group the mean is a coarse summary and the lie is worth 1.5. At 4.4 rows per group it is nearly the row's own target and the lie is worth **118.0** — a **17.4%** apparent improvement that does not exist.

Read the bottom row again

For `yr×mnth×weekday` (168 groups, 4.4 rows each):

no group feature at all	660.5
honest (computed in-fold)	678.1
leaky (computed on all data)	560.1

The honest feature is the **worst** option on the table — it makes the model worse than not having it. The leaky one looks like the **best** feature in the whole deck.

The more attractive the number, the harder you should look at it.

You will not catch this by staring at cross-validation scores — leakage *improves* them. You catch it by knowing how the feature was built.

Doing it right: out-of-fold encoding

The fix is mechanical. For each fold, compute the statistic on the **other** folds only:

```
for train_idx, val_idx in KFold(5, shuffle=True, random_state=509).split(X):
    means = y[train_idx].groupby(key[train_idx]).mean()      # fit on train
                    part ONLY
    f_train = key[train_idx].map(means)
    f_val    = key[val_idx].map(means).fillna(y[train_idx].mean()) # unseen
                    group -> global
```

- ▶ **Unseen groups** are guaranteed at prediction time. Fall back to the global mean.
- ▶ **Smooth small groups** toward the global mean: $\tilde{\mu}_g = \frac{n_g \mu_g + m \mu}{n_g + m}$, with m a prior weight. A group of 2 rows should not be trusted like a group of 200.
- ▶ Libraries: `category_encoders` (TargetEncoder, cross-validated), and `sklearn.preprocessing.TargetEncoder`.

Get it into the Pipeline, or it will leak anyway

Unenforceable if you engineer in a notebook and cross-validate afterwards. Make the feature a **transformer**, so fit only ever sees the training fold.

```
# stateless (ratios, differences): nothing learned -> safe anywhere
ratio = FunctionTransformer(lambda X: np.c_[X, X[:, 0] * X[:, 1]])

# stateful (group means, bin edges): MUST be refitted per fold
class GroupMean(BaseEstimator, TransformerMixin):
    def fit(self, X, y):
        self.mu_ = pd.Series(y).groupby(X["season"]).mean()
        self.default_ = y.mean()
        return self
    def transform(self, X):
        g = X["season"].map(self.mu_).fillna(self.default_)
        return X.assign(grp_mean=g)

pipe = Pipeline([("f", GroupMean()), ("sc", StandardScaler()), ("m", Ridge())
                ])
cross_val_score(pipe, X, y, cv=5)      # refit inside every fold
```

The test: does your feature *learn* anything from the data — a mean, a category list, bin edges? If yes, it belongs inside a Pipeline.

Handed forward: lags, rolling windows, forward splits



Everything in this section assumed rows are **exchangeable** — that any row could have been in the training set.

When rows are ordered in *time*, aggregation gets a second rule: **only look backwards**.

- ▶ **Lags** — `y.shift(1)`, `shift(7)`: yesterday, last week.
- ▶ **Rolling windows** — `y.shift(1).rolling(7).mean()`. Note the *shift before* the rolling: without it, the window contains the value you are trying to predict.
- ▶ **Forward splits** — `TimeSeriesSplit`, never a random split.

Next chapters: this is the spine of the time-series chapter, where it gets a full treatment with forecasting horizons and the naive baseline. We are deliberately not doing it twice.

The information is not in your table

Where feature engineering still beats any model, and always will.

Joins: the features no model can invent

Go back to slide 2. `holiday` and all five weather columns **are joins**. Somebody decided the weather mattered, went and got it, and merged it on the date.

What to join:

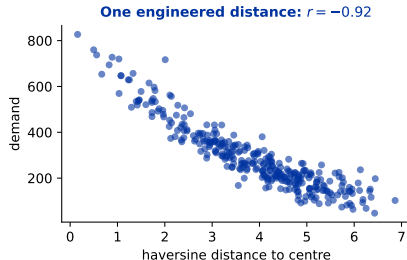
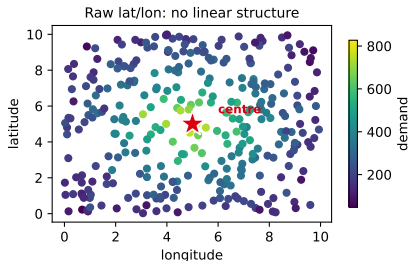
- ▶ calendar: holidays, school terms, paydays
- ▶ weather: temperature, precipitation, wind
- ▶ demographics: income and density by area
- ▶ reference data: official price indices, exchange rates

Point-in-time correctness. Join the value *as it was known on that date*, not today's revised figure. Economic statistics get revised for years; joining the final revision to a historical row is leakage from the future.

This is the highest-leverage thing in the deck and the one place where “good features beat a better algorithm” is still unambiguously true — because no amount of gradient boosting recovers information that is not in your table.

Geospatial features

Illustration on a synthetic city - bike-day.csv has no coordinates



Raw latitude and longitude are nearly useless to a linear model — “demand rises with latitude” is rarely true. The *relationships* are what predict.

- **distances** to landmarks (centre, metro, river)
- **cluster identifiers** from k-means on coordinates
- **grid cells** — geohash, H3 — then aggregate per cell
- **counts of points of interest (POI)** within a radius

Different data, same idea: bike-day.csv is a daily system-wide total with no coordinates, so this is a synthetic illustration.

High-cardinality categoricals: crosses

Lecture 3 gave you target and frequency encoding for a column with hundreds of levels. The new idea is combining two of them:

```
df["district_x_rooms"] = df["district"] + "_" + df["rooms"].astype(str)    #  
    "Kentron_3br"
```

- ▶ Captures effects that are **jointly** true: three-room flats are expensive *in Kentron specifically*.
- ▶ Heavily used in click-through rate (CTR) prediction — Google's Wide & Deep model is built on exactly this idea.
- ▶ **The risk is explosion:** 100 districts $\times$ 10 room counts = 1000 levels, most with a handful of rows each. Every one of them then needs encoding, and the small ones need smoothing (previous section).
- ▶ **The modern alternative** is to learn an *embedding* per category — a short dense vector, fitted with the model. Neural networks do this natively.

Text, in one slide

Text has to become numbers. Three layers, increasing in power and cost:

1. **Engineered scalars.** Length, word count, punctuation, capital-letter ratio, digit count. Cheap, and surprisingly strong for spam and sentiment.
2. **Bag-of-words and term frequency–inverse document frequency (TF-IDF).** Count the words, down-weight the ones that appear everywhere. The classical workhorse.
3. **Embeddings.** Learned dense vectors — deferred to the language chapter.

```
tfidf = TfidfVectorizer(max_features=2000, ngram_range=(1, 2), min_df=5)
X_text = tfidf.fit_transform(df["description"])           # fit on TRAIN only

df["desc_len"]     = df["description"].str.len()          # the cheap layer
                    often
df["caps_ratio"]   = df["description"].str.count(r"[A-Z]") / df["desc_len"]
                    # gets you most of it
```

Note: bike-day has no text either — illustration only.

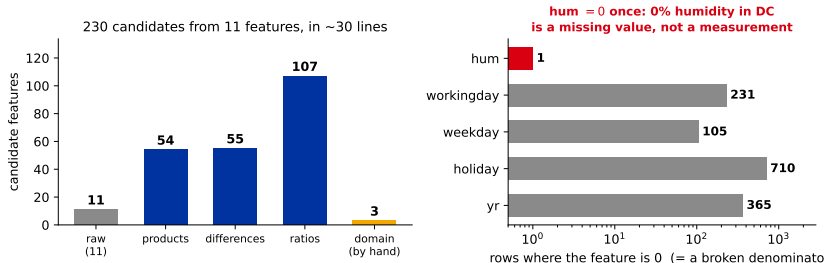
Now do it two hundred times

Generate cheaply — because next lecture prunes ruthlessly.

Systematic expansion

Stop hand-picking. Apply this deck's operations *mechanically* to every pair of columns:

```
for a, b in itertools.combinations(FEATURES, 2):  
    pool[f"{a}_x_{b}"] = df[a] * df[b]           # 55 products  
    pool[f"{a}_minus_{b}"] = df[a] - df[b]       # 55 differences  
for a, b in itertools.permutations(FEATURES, 2):  
    pool[f"{a}_over_{b}"] = df[a] / df[b]       # 110 ratios, both ways
```



231 columns from 11, in about thirty lines — 3 constant and 1 duplicated column drop out, leaving **230**. Most are meaningless. That is next lecture's problem.

Expansion is also a data-quality probe

Those 110 ratios produced **14,120 non-finite cells**. Five features contain zeros, and dividing by them gives $\pm\infty$:

holiday	710	legitimate — it is a flag
yr	365	legitimate — 2011 is coded 0
workingday	231	legitimate — a flag
weekday	105	legitimate — Sunday is 0
hum	1	not legitimate

hum = 0 on **2011-03-10**. Zero percent humidity in Washington DC in March is **physically impossible**. It is an unflagged missing value.

Nobody found this row by looking at the data — it was found by **building a feature**. Four lectures of interpretability ran on this table without it surfacing. Mechanical expansion is a surprisingly good bug detector.

Automated feature engineering: the libraries

You are not the first person to think “a machine could do this”.

- ▶ **featuretools** — Deep Feature Synthesis (DFS) walks a *relational* schema and stacks aggregations across table joins: `MEAN(rentals.duration)`, `COUNT(stations.rentals)`, `STD(users.trips)`.
- ▶ **tsfresh** — hundreds of statistical features from time series (autocorrelations, spectral peaks, entropy), with a built-in significance filter.
- ▶ **autofeat**, **featurewiz** — general-purpose automated construction and screening.

Where these shine is many related tables — customers, orders, order lines — which is exactly what our single flat `bike-day.csv` is not. On one table, DFS can only re-derive the transformations we just wrote by hand.

Use them for exploration, not production. Auto-generated features are hard to maintain, expensive to serve, and usually lose to a handful of domain features — which is the whole argument of slide 7.

Five ways to shoot yourself

Four of them make your scores go *up*.

Five feature-engineering pitfalls

- ✓ 1. **Target leakage.** A feature built from the target, computed before the split. Three appearances in this deck: `casual+registered`, `group means`, and `target encoding`. *Fix:* out-of-fold, inside a Pipeline.
- ✓ 2. **Look-ahead bias.** Using future values in a “rolling” or aggregate feature. *Fix:* trailing windows only, `shift` before rolling. (Time-series chapter.)
3. **Train-serve skew.** A feature that is easy on historical data and impossible at prediction time — “total lifetime purchases” when the customer is still shopping. *Fix:* ask “would I know this value at 9am on the day I have to predict?”
4. **Hard-coded statistics.** “Just divide by the mean.” Whose mean? Recomputing it on test data is leakage; hard-coding it from a notebook is a bug waiting for the next retrain. *Fix:* fit it, store it, ship it in the pipeline.
- ✓ 5. **Over-engineering.** 500 features on 1000 rows. Multicollinearity, variance, and the curse of dimensionality all bite at once. *Fix:* next lecture.

Numbers 1, 2 and 4 all make your cross-validation score *better*. That is what makes them dangerous — you have no reason to go looking.

Recap

- ▶ **Somebody made your features.** Of 16 columns in `bike-day.csv`, one is raw. The rest are date parts, two joins, and a combination.
- ▶ **Feature engineering is worth +81.5 MAE to Ridge and -4.3 to a forest.** It pays where the model cannot get there alone: joins, domain knowledge, small data, time features, linear models.
- ▶ **Three sources:** the same row (ratios, interactions, bins), many rows (aggregations), outside the data (joins, geography, text).
- ▶ **Biggest single win here was binning** (+59.0), because temp's effect saturates and a straight line cannot.
- ▶ **Leakage scales with group precision** — 1.5 MAE at 183 rows per group, 118.0 at 4.4. The honest feature was worse than no feature; the leaky one looked best.
- ▶ **If a feature learns anything from data, it goes inside the Pipeline.**
- ▶ **A corrupt row** (`hum = 0` in March) was found by *building features*, not by inspecting data.

Next lecture: you can now generate 230 features in thirty lines. Which fifteen do you keep? Every importance tool from chapter 5 gives you a *ranking* — and none of them tells you where to *cut*.